Project Atlas

Azure Medallion Lakehouse

Real-Time Financial Transaction Pipeline

Author: Edem

Date: April 2026

Version: 1.0

Stack:	Apache Kafka, Spark Structured Streaming Delta Lake, MinIO, MongoDB, PostgreSQL
Layers:	Bronze → Silver → Gold
Azure mapping:	Event Hubs, ADLS Gen2, Databricks Cosmos DB, Synapse Analytics
CI/CD:	Azure DevOps Pipelines Bicep IaC templates

Kraków, Poland

Contents

1 Project Overview **2**
- 1.1 What We Built . . . 2
- 1.2 Architecture Overview . . . 2
- 1.3 Technology Stack . . . 2
- 1.4 Bank Simulator Configuration . . . 3

2 Local Development Environment **4**
- 2.1 Prerequisites . . . 4
- 2.2 Project Structure . . . 4
- 2.3 Environment Variables . . . 4
- 2.4 Docker Compose Stack . . . 5
- 2.5 Custom Spark Dockerfile . . . 8

3 Bronze Layer — Streaming Ingestion **9**
- 3.1 Overview . . . 9
- 3.2 Spark Session Configuration . . . 9
- 3.3 Bronze Schema . . . 10
- 3.4 Kafka Stream Reader . . . 10
- 3.5 Bronze Delta Lake Write . . . 11

4 Silver Layer — Stream Processing **12**
- 4.1 Overview . . . 12
- 4.2 Validation Rules . . . 12
- 4.3 Fraud Signals . . . 12
- 4.4 Silver Transformation Code . . . 13
- 4.5 foreachBatch Pattern . . . 14

5 Gold Layer — Aggregation **16**
- 5.1 Overview . . . 16
- 5.2 Gold Schema . . . 16
- 5.3 Gold Aggregations . . . 17
- 5.4 Idempotent PostgreSQL Write . . . 17

6 Azure Deployment **19**
- 6.1 Azure Mapping . . . 19
- 6.2 Path Changes for Azure . . . 19
- 6.3 Bicep Infrastructure Template . . . 20
- 6.4 Azure DevOps Pipeline . . . 21

7 Operations **24**
7.1 Startup and Shutdown 24
7.2 Service URLs 24
7.3 Notebook Execution Order 24
7.4 Monitoring Queries 25

8 Git Repository Structure **27**
8.1 .gitignore 27
8.2 Git Workflow 27

9 Interview Reference **29**
9.1 Key Talking Points 29
9.2 Architecture Decisions 29

1. Project Overview

1.1 What We Built

Project Atlas is a production-grade real-time financial transaction pipeline built on the medallion lakehouse architecture. It simulates three Polish banks — PKO Bank Polski, mBank, and ING Bank — streaming transactions simultaneously through a Bronze, Silver, and Gold processing pipeline.

The project demonstrates the complete data engineering lifecycle: local development with Docker, streaming ingestion with Kafka and Spark Structured Streaming, data quality enforcement with Delta Lake, fraud detection with configurable signal computation, and business metric aggregation for Power BI consumption.

1.2 Architecture Overview

Listing 1.1: Complete Pipeline Flow

```
Bank Simulators (PKO / mBank / ING)
        |
        v  Kafka topics (one per bank)
Apache Kafka Broker
        |
        v  Spark Structured Streaming
Bronze Delta Lake  (s3a://bronze/delta/transactions/)
        |
        v  Spark Structured Streaming
Silver Delta Lake  (s3a://silver/delta/transactions/)
        |
        v  Spark Batch Aggregation
Gold PostgreSQL    (gold.daily_customer_spending)
                   (gold.fraud_indicators)
                   (gold.daily_merchant_summary)
        |
        v
Power BI Dashboard
```

1.3 Technology Stack

Local Service	Azure Equivalent	Purpose
Apache Kafka	Azure Event Hubs	Message broker
Apache Spark	Azure Databricks	Data processing

MinIO	ADLS Gen2	Object storage
MongoDB	Azure Cosmos DB	Audit logging
PostgreSQL	Azure Synapse	Gold SQL layer
JupyterLab	Databricks notebooks	Development
Zookeeper	Not needed	Kafka coordination
Docker Compose	Bicep + AKS	Infrastructure

1.4 Bank Simulator Configuration

Bank	**Topic**	**Interval**	**Fraud Rate**
PKO Bank Polski	bank.pko.transactions	2 seconds	5%
mBank	bank.mbank.transactions	3 seconds	8%
ING Bank Slaski	bank.ing.transactions	1.5 seconds	3%
Combined throughput		~90 transactions/minute	

2. Local Development Environment

2.1 Prerequisites

- Docker Desktop 4.x or later
- 16GB RAM recommended (8GB minimum)
- 20GB free disk space
- macOS, Linux, or Windows with WSL2

2.2 Project Structure

Listing 2.1: Directory Structure

```
project-atlas/
  docker-compose.yml          # Full stack definition
  Dockerfile.spark            # Custom Spark with Kafka JARs
  .env                        # Secrets (never commit to Git)
  .env.example                # Template for .env
  .gitignore                  # Excludes .env and generated files
  requirements.txt            # Python dependencies
  spark-jars/                 # Pre-downloaded Kafka JARs
  notebooks/
    02_stream_bronze_ingestion.ipynb
    03_stream_silver_processing.ipynb
    04_gold_aggregation.ipynb
  src/
    simulators/
      bank_simulator.py       # Bank transaction producer
      requirements.txt        # kafka-python-ng==2.2.2
      Dockerfile
    sql/
      001_create_gold_schema.sql
```

2.3 Environment Variables

Listing 2.2: .env file

```
# MinIO - local Azure Data Lake Storage
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=atlasadmin
MINIO_SECRET_KEY=atlaspassword123

# Bucket names
```

```
BRONZE_BUCKET=bronze
SILVER_BUCKET=silver
GOLD_BUCKET=gold
QUARANTINE_BUCKET=quarantine

# MongoDB - local Cosmos DB
MONGODB_URI=mongodb://atlasadmin:atlaspassword123@mongodb:27017
MONGODB_DB=atlas_pipeline

# PostgreSQL - local Synapse Analytics
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=atlas_gold
POSTGRES_USER=atlasadmin
POSTGRES_PASSWORD=atlaspassword123

# Pipeline settings
PIPELINE_NAME=atlas-gold-aggregation
```

2.4 Docker Compose Stack

Listing 2.3: docker-compose.yml (key services)

```
version: "3.8"

networks:
  atlas-network:
    driver: bridge

volumes:
  minio-data:
  mongodb-data:
  postgres-data:

services:
  spark-master:
    build:
      context: .
      dockerfile: Dockerfile.spark
    container_name: atlas-spark-master
    command: /opt/spark/bin/spark-class
             org.apache.spark.deploy.master.Master
    environment:
      - SPARK_MASTER_HOST=spark-master
      - SPARK_MASTER_PORT=7077
      - SPARK_MASTER_WEBUI_PORT=8080
    ports:
      - "8080:8080"
      - "7077:7077"
    networks:
      - atlas-network

  spark-worker:
    build:
      context: .
      dockerfile: Dockerfile.spark
    container_name: atlas-spark-worker
```

```
    command: /opt/spark/bin/spark-class
             org.apache.spark.deploy.worker.Worker
             spark://spark-master:7077
    environment:
      - SPARK_WORKER_MEMORY=2g
      - SPARK_WORKER_CORES=2
    networks:
      - atlas-network

  jupyter:
    image: jupyter/all-spark-notebook:latest
    container_name: atlas-jupyter
    env_file:
      - .env
    environment:
      - JUPYTER_TOKEN=atlas2024
    ports:
      - "8888:8888"
    networks:
      - atlas-network

  minio:
    image: minio/minio:latest
    container_name: atlas-minio
    command: server /data --console-address ":9001"
    environment:
      - MINIO_ROOT_USER=atlasadmin
      - MINIO_ROOT_PASSWORD=atlaspassword123
    ports:
      - "9000:9000"
      - "9001:9001"
    networks:
      - atlas-network

  mongodb:
    image: mongo:7.0
    container_name: atlas-mongodb
    environment:
      - MONGO_INITDB_ROOT_USERNAME=atlasadmin
      - MONGO_INITDB_ROOT_PASSWORD=atlaspassword123
    ports:
      - "27017:27017"
    networks:
      - atlas-network

  postgres:
    image: postgres:16
    container_name: atlas-postgres
    environment:
      - POSTGRES_USER=atlasadmin
      - POSTGRES_PASSWORD=atlaspassword123
      - POSTGRES_DB=atlas_gold
    ports:
      - "5432:5432"
    networks:
      - atlas-network
    volumes:
      - ./src/sql:/docker-entrypoint-initdb.d
```

```
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: atlas-zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    networks:
      - atlas-network

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: atlas-kafka
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS:
        PLAINTEXT://0.0.0.0:29092,
        PLAINTEXT_HOST://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS:
        PLAINTEXT://kafka:29092,
        PLAINTEXT_HOST://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
      KAFKA_NUM_PARTITIONS: 3
    ports:
      - "9092:9092"
    networks:
      - atlas-network

  bank-pko:
    build:
      context: ./src/simulators
    environment:
      - KAFKA_BROKER=kafka:29092
      - KAFKA_TOPIC=bank.pko.transactions
      - BANK_CODE=PKO
      - SEND_INTERVAL=2.0
      - FRAUD_RATE=0.05
    networks:
      - atlas-network
    restart: unless-stopped

  bank-mbank:
    build:
      context: ./src/simulators
    environment:
      - KAFKA_BROKER=kafka:29092
      - KAFKA_TOPIC=bank.mbank.transactions
      - BANK_CODE=MBANK
      - SEND_INTERVAL=3.0
      - FRAUD_RATE=0.08
    networks:
      - atlas-network
    restart: unless-stopped

  bank-ing:
    build:
      context: ./src/simulators
    environment:
```

```
      - KAFKA_BROKER=kafka:29092
      - KAFKA_TOPIC=bank.ing.transactions
      - BANK_CODE=ING
      - SEND_INTERVAL=1.5
      - FRAUD_RATE=0.03
    networks:
      - atlas-network
    restart: unless-stopped
```

2.5 Custom Spark Dockerfile

Listing 2.4: Dockerfile.spark

```
FROM apache/spark:3.5.1

USER root

# Kafka JARs - for reading from Kafka topics
RUN curl -o /opt/spark/jars/spark-sql-kafka-0-10_2.12-3.5.0.jar \
    https://repo1.maven.org/maven2/org/apache/spark/\
spark-sql-kafka-0-10_2.12/3.5.0/\
spark-sql-kafka-0-10_2.12-3.5.0.jar && \
    curl -o /opt/spark/jars/kafka-clients-3.4.1.jar \
    https://repo1.maven.org/maven2/org/apache/kafka/\
kafka-clients/3.4.1/kafka-clients-3.4.1.jar && \
    curl -o /opt/spark/jars/spark-token-provider-kafka\
-0-10_2.12-3.5.0.jar \
    https://repo1.maven.org/maven2/org/apache/spark/\
spark-token-provider-kafka-0-10_2.12/3.5.0/\
spark-token-provider-kafka-0-10_2.12-3.5.0.jar && \
    curl -o /opt/spark/jars/commons-pool2-2.11.1.jar \
    https://repo1.maven.org/maven2/org/apache/commons/\
commons-pool2/2.11.1/commons-pool2-2.11.1.jar

# Hadoop AWS JARs - for S3A filesystem (MinIO)
RUN curl -o /opt/spark/jars/hadoop-aws-3.3.4.jar \
    https://repo1.maven.org/maven2/org/apache/hadoop/\
hadoop-aws/3.3.4/hadoop-aws-3.3.4.jar && \
    curl -o /opt/spark/jars/aws-java-sdk-bundle-1.12.367.jar \
    https://repo1.maven.org/maven2/com/amazonaws/\
aws-java-sdk-bundle/1.12.367/\
aws-java-sdk-bundle-1.12.367.jar

USER spark
```

3. Bronze Layer — Streaming Ingestion

3.1 Overview

The Bronze layer is the raw landing zone. Every transaction from all three Kafka topics lands here exactly as received — no transformations, no cleaning, no filtering. All columns are StringType. Bad data is stored alongside good data. The Bronze layer is immutable.

Key principle: Bronze stores everything, trusts nothing.

3.2 Spark Session Configuration

Listing 3.1: Bronze Spark Session

```python
import os
from pyspark.sql import SparkSession

os.environ["PYSPARK_SUBMIT_ARGS"] = (
    "--packages "
    "io.delta:delta-spark_2.12:3.1.0,"
    "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0,"
    "org.apache.hadoop:hadoop-aws:3.3.4,"
    "com.amazonaws:aws-java-sdk-bundle:1.12.367,"
    "org.apache.spark:spark-token-provider-kafka-0-10_2.12:3.5.0,"
    "org.apache.commons:commons-pool2:2.11.1 "
    "pyspark-shell"
)

spark = (
    SparkSession.builder
    .appName("Atlas - Stream Bronze Ingestion")
    .master("local[2]")
    .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.shuffle.partitions", "4")
    .config("spark.hadoop.fs.s3a.endpoint",
            CONFIG["minio_endpoint"])
    .config("spark.hadoop.fs.s3a.access.key",
            CONFIG["minio_access_key"])
    .config("spark.hadoop.fs.s3a.secret.key",
            CONFIG["minio_secret_key"])
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled",
            "false")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
    .config("spark.hadoop.fs.s3a.impl",
```

```
            "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
            "org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
    .getOrCreate()
)
```

3.3 Bronze Schema

Listing 3.2: Bronze Schema Definition

```
transaction_schema = StructType([
    StructField("transaction_id",    StringType(), True),
    StructField("customer_id",       StringType(), True),
    StructField("account_id",        StringType(), True),
    StructField("merchant_id",       StringType(), True),
    StructField("merchant_name",     StringType(), True),
    StructField("merchant_category", StringType(), True),
    StructField("amount",            StringType(), True),
    StructField("currency",          StringType(), True),
    StructField("transaction_type",  StringType(), True),
    StructField("status",            StringType(), True),
    StructField("timestamp",         StringType(), True),
    StructField("bank_code",         StringType(), True),
    StructField("country",           StringType(), True),
    StructField("device_type",       StringType(), True),
    StructField("ip_address",        StringType(), True),
])
# All StringType - Bronze accepts everything as-is
# Silver handles type casting and validation
```

3.4 Kafka Stream Reader

Listing 3.3: Reading from Kafka

```
kafka_raw_stream = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", CONFIG["kafka_broker"])
    .option("subscribe", CONFIG["kafka_topics"])
    .option("startingOffsets", "latest")
    .option("failOnDataLoss", "false")
    .option("maxOffsetsPerTrigger", "100")
    .load()
)

# Parse JSON value bytes into structured columns
parsed_stream = (
    kafka_raw_stream
    .select(
        F.col("value").cast("string").alias("json_string"),
        F.col("topic").alias("kafka_topic"),
        F.col("partition").alias("kafka_partition"),
        F.col("offset").alias("kafka_offset"),
        F.col("timestamp").alias("kafka_timestamp"),
    )
    .withColumn("data",
```

```
        F.from_json(F.col("json_string"), transaction_schema))
    .select("data.*", "kafka_topic", "kafka_partition",
            "kafka_offset", "kafka_timestamp")
)
```

3.5 Bronze Delta Lake Write

Listing 3.4: Streaming Write to Bronze

```
bronze_stream = (
    parsed_stream
    .withColumn("ingested_at",     F.current_timestamp())
    .withColumn("pipeline_run_id", F.lit(RUN_ID))
    .withColumn("processing_date", F.current_date().cast("string"))
)

BRONZE_DELTA_PATH =
   f"s3a://{CONFIG['bronze_bucket']}/delta/transactions/"
CHECKPOINT_PATH   =
   f"s3a://{CONFIG['bronze_bucket']}/checkpoints/bronze-stream/"

bronze_query = (
    bronze_stream
    .writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", CHECKPOINT_PATH)
    .partitionBy("processing_date", "bank_code")
    .trigger(processingTime="10 seconds")
    .start(BRONZE_DELTA_PATH)
)
```

4. Silver Layer — Stream Processing

4.1 Overview

The Silver layer reads Bronze Delta Lake as a continuous stream and applies five transformation stages to every record:

1. **Validation** — 7 rules, quarantine failures
2. **Type casting** — String to Decimal, Timestamp
3. **Fraud signals** — 5 boolean flags + score
4. **Currency normalisation** — everything to EUR
5. **Silver metadata** — processed_at, run_id

Key principle: Silver validates everything. Bad records go to quarantine, never silently dropped.

4.2 Validation Rules

Rule	Field	Quarantine Reason
1	transaction_id not null	missing_transaction_id
2	customer_id not null	missing_customer_id
3	amount not null	missing_amount
4	amount is positive decimal	invalid_amount
5	currency in valid list	invalid_currency
6	bank_code recognised	unknown_bank_code
7	timestamp parseable	invalid_timestamp

4.3 Fraud Signals

Signal	Condition	Weight
is_large_amount	amount > EUR 5,000	1
is_suspicious_ip	IP in Tor exit node ranges	1
is_unknown_merchant	merchant_category = unknown	1
is_pending_status	status = PENDING	1
is_unusual_currency	currency not PLN/EUR/USD/GBP	1

fraud_score	Sum of all signals	0–5
risk_level	0–1 = low 2–3 = medium 4–5 = high	

4.4 Silver Transformation Code

Listing 4.1: Validation Function

```
def add_validation(df):
    return df.withColumn(
        "validation_error",
        F.when(
            F.col("transaction_id").isNull() |
            (F.trim(F.col("transaction_id")) == ""),
            F.lit("missing_transaction_id")
        ).when(
            F.col("amount").cast(DecimalType(15, 2)).isNull() |
            (F.col("amount").cast(DecimalType(15, 2)) <= 0),
            F.lit("invalid_amount")
        ).when(
            ~F.upper(F.trim(F.col("currency")))
             .isin(CONFIG["valid_currencies"]),
            F.lit("invalid_currency")
        ).when(
            ~F.upper(F.trim(F.col("bank_code")))
             .isin(CONFIG["valid_bank_codes"]),
            F.lit("unknown_bank_code")
        ).otherwise(F.lit(None).cast("string"))
    )
```

Listing 4.2: Fraud Signal Computation

```
def add_fraud_signals(df):
    ip_fraud_expr = F.lit(False)
    for prefix in CONFIG["fraud_ip_ranges"]:
        ip_fraud_expr = ip_fraud_expr | \
            F.col("ip_address").startswith(prefix)

    return (df
        .withColumn("is_large_amount",
            F.col("amount") > CONFIG["fraud_large_amount_eur"])
        .withColumn("is_suspicious_ip", ip_fraud_expr)
        .withColumn("is_unknown_merchant",
            F.col("merchant_category") == "unknown")
        .withColumn("is_pending_status",
            F.col("status") == "PENDING")
        .withColumn("is_unusual_currency",
            ~F.col("currency").isin(["PLN","EUR","USD","GBP"]))
        .withColumn("fraud_score",
            F.col("is_large_amount").cast("int") +
            F.col("is_suspicious_ip").cast("int") +
            F.col("is_unknown_merchant").cast("int") +
            F.col("is_pending_status").cast("int") +
            F.col("is_unusual_currency").cast("int"))
        .withColumn("risk_level",
```

```
            F.when(F.col("fraud_score") >= 4, F.lit("high"))
             .when(F.col("fraud_score") >= 2, F.lit("medium"))
             .otherwise(F.lit("low")))
    )
```

Listing 4.3: Currency Normalisation to EUR

```
def add_eur_amount(df):
    fx_rates = {
        "PLN": 0.23, "USD": 0.92, "GBP": 1.17,
        "AUD": 0.60, "SGD": 0.70, "EUR": 1.00,
    }
    eur_expr = F.when(F.col("currency") == "EUR", F.col("amount"))
    for currency, rate in fx_rates.items():
        if currency != "EUR":
            eur_expr = eur_expr.when(
                F.col("currency") == currency,
                F.round(F.col("amount") * F.lit(rate), 2)
            )
    return df.withColumn("amount_eur",
        eur_expr.otherwise(F.col("amount")).cast(DecimalType(15,2)))
```

4.5 foreachBatch Pattern

Listing 4.4: Silver foreachBatch Writer

```
def process_silver_batch(batch_df, batch_id):
    record_count = batch_df.count()
    if record_count == 0:
        return

    validated_df  = add_validation(batch_df)
    valid_df      = validated_df.filter(
        F.col("validation_error").isNull()).drop("validation_error")
    quarantine_df = validated_df.filter(
        F.col("validation_error").isNotNull())

    silver_df = cast_types(valid_df)
    silver_df = add_fraud_signals(silver_df)
    silver_df = add_eur_amount(silver_df)
    silver_df = add_silver_metadata(silver_df)

    # Write valid to Silver Delta Lake
    silver_df.write.format("delta").mode("append") \
        .option("mergeSchema", "true") \
        .partitionBy("processing_date", "bank_code") \
        .save(SILVER_DELTA_PATH)

    # Write invalid to quarantine
    quarantine_df.withColumn("quarantine_reason",
        F.col("validation_error")) \
        .write.format("json").mode("append") \
        .save(QUARANTINE_PATH)

silver_query = (
    bronze_stream.writeStream
    .foreachBatch(process_silver_batch)
```

```
    .outputMode("update")
    .option("checkpointLocation", CHECKPOINT_PATH)
    .trigger(processingTime="30 seconds")
    .start()
)
```

5. Gold Layer — Aggregation

5.1 Overview

The Gold layer reads Silver Delta Lake as a batch job and produces pre-aggregated business metrics written to PostgreSQL. It runs every 5 minutes and uses a delete-first idempotent write pattern to prevent duplicates.

5.2 Gold Schema

Listing 5.1: PostgreSQL Gold Schema

```sql
CREATE SCHEMA IF NOT EXISTS gold;

CREATE TABLE gold.daily_customer_spending (
    id                      SERIAL PRIMARY KEY,
    transaction_date        DATE          NOT NULL,
    customer_id             VARCHAR(20)   NOT NULL,
    bank_code               VARCHAR(20)   NOT NULL,
    total_amount_eur        DECIMAL(15,2),
    transaction_count       INTEGER,
    avg_transaction_eur     DECIMAL(15,2),
    max_transaction_eur     DECIMAL(15,2),
    grocery_spend_eur       DECIMAL(15,2) DEFAULT 0,
    ecommerce_spend_eur     DECIMAL(15,2) DEFAULT 0,
    fuel_spend_eur          DECIMAL(15,2) DEFAULT 0,
    entertainment_spend_eur DECIMAL(15,2) DEFAULT 0,
    transport_spend_eur     DECIMAL(15,2) DEFAULT 0,
    top_category            VARCHAR(50),
    pipeline_run_id         VARCHAR(50),
    created_at              TIMESTAMP     DEFAULT NOW(),
    UNIQUE(transaction_date, customer_id, bank_code)
);

CREATE TABLE gold.fraud_indicators (
    id                  SERIAL PRIMARY KEY,
    transaction_id      VARCHAR(50)   NOT NULL UNIQUE,
    customer_id         VARCHAR(20)   NOT NULL,
    transaction_date    DATE          NOT NULL,
    amount_eur          DECIMAL(15,2),
    is_large_amount     BOOLEAN       DEFAULT FALSE,
    is_suspicious_ip    BOOLEAN       DEFAULT FALSE,
    is_unknown_merchant BOOLEAN       DEFAULT FALSE,
    is_pending_status   BOOLEAN       DEFAULT FALSE,
    is_unusual_currency BOOLEAN       DEFAULT FALSE,
    fraud_score         INTEGER       DEFAULT 0,
    risk_level          VARCHAR(10),
```

```
    pipeline_run_id      VARCHAR(50),
    created_at           TIMESTAMP      DEFAULT NOW()
);

CREATE TABLE gold.daily_merchant_summary (
    id                   SERIAL PRIMARY KEY,
    transaction_date     DATE           NOT NULL,
    merchant_id          VARCHAR(50)    NOT NULL,
    merchant_name        VARCHAR(200),
    merchant_category    VARCHAR(50),
    total_revenue_eur    DECIMAL(15,2),
    transaction_count    INTEGER,
    unique_customers     INTEGER,
    avg_transaction_eur  DECIMAL(15,2),
    pipeline_run_id      VARCHAR(50),
    created_at           TIMESTAMP      DEFAULT NOW(),
    UNIQUE(transaction_date, merchant_id)
);
```

5.3 Gold Aggregations

Listing 5.2: Daily Customer Spending Aggregation

```
daily_spending_df = (
    silver_df
    .groupBy("processing_date", "customer_id", "bank_code")
    .agg(
        F.round(F.sum("amount_eur"),  2).alias("total_amount_eur"),
        F.count("*").alias("transaction_count"),
        F.round(F.avg("amount_eur"),
            2).alias("avg_transaction_eur"),
        F.round(F.max("amount_eur"),
            2).alias("max_transaction_eur"),
        F.round(F.sum(
            F.when(F.col("merchant_category") == "grocery",
                   F.col("amount_eur")).otherwise(0)
        ), 2).alias("grocery_spend_eur"),
        F.first("merchant_category").alias("top_category"),
        F.lit(RUN_ID).alias("pipeline_run_id"),
        F.current_timestamp().alias("computed_at")
    )
)
```

5.4 Idempotent PostgreSQL Write

Listing 5.3: Delete-First Upsert Pattern

```
def delete_todays_records(table: str):
    conn = psycopg2.connect(
        host=CONFIG["postgres_host"],
        port=CONFIG["postgres_port"],
        dbname=CONFIG["postgres_db"],
        user=CONFIG["postgres_user"],
        password=CONFIG["postgres_password"]
    )
    try:
```

```
        cur = conn.cursor()
        cur.execute(
            f"DELETE FROM {table} WHERE transaction_date = %s::date",
            (CONFIG["processing_date"],)
        )
        conn.commit()
    finally:
        conn.close()

# Write pattern for each Gold table:
# 1. Delete today's existing records
# 2. Append fresh aggregations
delete_todays_records("gold.daily_customer_spending")
spending_pg.write.jdbc(
    url=POSTGRES_JDBC_URL,
    table="gold.daily_customer_spending",
    mode="append",
    properties=POSTGRES_PROPS
)
```

6. Azure Deployment

6.1 Azure Mapping

Every local Docker service maps directly to a managed Azure service. The Spark and Delta Lake code is identical — only connection strings change.

Local	Azure	Code change
Apache Kafka	Azure Event Hubs	Connection string only
MinIO (s3a://)	ADLS Gen2 (abfss://)	Path prefix only
Spark local[2]	Azure Databricks	Remove .master()
MongoDB	Azure Cosmos DB	Connection string only
PostgreSQL	Azure Synapse	Connection string only
.env file	Azure Key Vault	dbutils.secrets.get()
docker-compose.yml	Bicep templates	New format
Manual notebook run	Azure Data Factory	Automated trigger
GitHub Actions	Azure DevOps	New YAML format

6.2 Path Changes for Azure

Listing 6.1: Local vs Azure Path Changes

```
# Local - MinIO with S3A protocol
BRONZE_PATH = "s3a://bronze/delta/transactions/"

# Azure - ADLS Gen2 with ABFSS protocol
BRONZE_PATH = (
    "abfss://bronze"
    "@atlasstorage.dfs.core.windows.net"
    "/delta/transactions/"
)

# Local - Kafka broker
.option("kafka.bootstrap.servers", "kafka:29092")

# Azure - Event Hubs (Kafka-compatible)
.option("kafka.bootstrap.servers",
        "atlas.servicebus.windows.net:9093")
.option("kafka.security.protocol",   "SASL_SSL")
.option("kafka.sasl.mechanism",      "PLAIN")
.option("kafka.sasl.jaas.config",
        "org.apache.kafka.common.security.plain.PlainLoginModule"
        " required username=\"$ConnectionString\""
        " password=\"Endpoint=sb://...\";")
```

6.3 Bicep Infrastructure Template

Listing 6.2: Main Bicep Template (main.bicep)

```
param location string = 'westeurope'
param projectName string = 'atlas'
param environment string = 'dev'

// Storage Account - ADLS Gen2
resource storageAccount
    'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: '${projectName}${environment}storage'
  location: location
  kind: 'StorageV2'
  sku: {
    name: 'Standard_LRS'
  }
  properties: {
    isHnsEnabled: true        // Enable hierarchical namespace
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
  }
}

// Bronze, Silver, Gold containers
resource bronzeContainer
    'Microsoft.Storage/storageAccounts/blobServices/containers@2023-01-01'
    = {
  name: '${storageAccount.name}/default/bronze'
  properties: {
    publicAccess: 'None'
  }
}

// Event Hubs Namespace - Kafka-compatible broker
resource eventHubsNamespace
    'Microsoft.EventHub/namespaces@2022-01-01-preview' = {
  name: '${projectName}-${environment}-eventhubs'
  location: location
  sku: {
    name: 'Standard'
    tier: 'Standard'
    capacity: 1
  }
  properties: {
    kafkaEnabled: true      // Kafka protocol support
  }
}

// Cosmos DB - MongoDB API
resource cosmosAccount
    'Microsoft.DocumentDB/databaseAccounts@2023-04-15' = {
  name: '${projectName}-${environment}-cosmos'
  location: location
  kind: 'MongoDB'
  properties: {
    databaseAccountOfferType: 'Standard'
    locations: [
      {
```

```
        locationName: location
        failoverPriority: 0
      }
    ]
  }
}

// Azure Database for PostgreSQL
resource postgresServer
    'Microsoft.DBforPostgreSQL/flexibleServers@2022-12-01' = {
  name: '${projectName}-${environment}-postgres'
  location: location
  sku: {
    name: 'Standard_D2s_v3'
    tier: 'GeneralPurpose'
  }
  properties: {
    version: '16'
    administratorLogin: 'atlasadmin'
    administratorLoginPassword:
        keyVaultRef.getSecret('postgres-password')
  }
}

// Key Vault for secrets
resource keyVault 'Microsoft.KeyVault/vaults@2023-02-01' = {
  name: '${projectName}-${environment}-kv'
  location: location
  properties: {
    sku: {
      family: 'A'
      name: 'standard'
    }
    tenantId: subscription().tenantId
    enableSoftDelete: true
    enabledForTemplateDeployment: true
  }
}
```

6.4 Azure DevOps Pipeline

Listing 6.3: azure-pipelines.yml

```
trigger:
  branches:
    include:
      - main
      - develop

variables:
  - group: atlas-variable-group

stages:
  - stage: Validate
    displayName: Validate
    jobs:
      - job: LintAndTest
```

```
        displayName: Lint and Test
        pool:
          vmImage: ubuntu-latest
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: '3.12'

          - script: |
              pip install flake8 pytest
              flake8 src/ --max-line-length=100
            displayName: Lint Python

          - task: AzureCLI@2
            displayName: Validate Bicep
            inputs:
              azureSubscription: atlas-service-connection
              scriptType: bash
              scriptLocation: inlineScript
              inlineScript: |
                az bicep build --file infrastructure/main.bicep

  - stage: DeployDev
    displayName: Deploy Dev
    dependsOn: Validate
    condition: succeeded()
    jobs:
      - deployment: DeployInfrastructure
        displayName: Deploy Infrastructure
        pool:
          vmImage: ubuntu-latest
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureResourceManagerTemplateDeployment@3
                  displayName: Deploy Bicep
                  inputs:
                    deploymentScope: Resource Group
                    azureResourceManagerConnection: atlas-service-
                       connection
                    resourceGroupName: atlas-dev-rg
                    location: westeurope
                    templateLocation: Linked artifact
                    csmFile: infrastructure/main.bicep
                    deploymentMode: Incremental

  - stage: DeployProd
    displayName: Deploy Production
    dependsOn: DeployDev
    condition: succeeded()
    jobs:
      - deployment: DeployProduction
        displayName: Deploy Production
        pool:
          vmImage: ubuntu-latest
        environment: production
```

```
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureResourceManagerTemplateDeployment@3
                  inputs:
                    resourceGroupName: atlas-prod-rg
                    csmFile: infrastructure/main.bicep
```

7. Operations

7.1 Startup and Shutdown

Listing 7.1: Daily Operations

```
# Start full stack
cd project-atlas
docker compose up -d

# Wait for Kafka to be ready (30 seconds)
sleep 30

# Check all 12 containers are running
docker compose ps

# Stop bank simulators (to save disk space)
docker compose stop bank-pko bank-mbank bank-ing

# Start bank simulators
docker compose start bank-pko bank-mbank bank-ing

# Clean shutdown
docker compose down

# Clean shutdown preserving volumes (data survives)
docker compose down --volumes=false
```

7.2 Service URLs

Service	URL	Credentials
JupyterLab	http://localhost:8888	token: atlas2024
Spark Master UI	http://localhost:8080	—
Spark Worker UI	http://localhost:8081	—
Spark App UI	http://localhost:4040	—
MinIO Console	http://localhost:9001	atlasadmin / atlaspassword123
Kafka UI	http://localhost:8090	—
PostgreSQL	localhost:5432	atlasadmin / atlaspassword123
MongoDB	localhost:27017	atlasadmin / atlaspassword123

7.3 Notebook Execution Order

Listing 7.2: Notebook Run Order

```
# After starting Docker stack:

# 1. Bronze streaming (keep running)
# Open: http://localhost:8888
# Run: 02_stream_bronze_ingestion.ipynb (all cells)
# Stream runs continuously every 10 seconds

# 2. Silver streaming (keep running)
# Open new browser tab
# Run: 03_stream_silver_processing.ipynb (all cells)
# Stream runs continuously every 30 seconds

# 3. Gold batch (run on demand or schedule)
# Run: 04_gold_aggregation.ipynb (all cells)
# Batch job - rerun every 5 minutes for fresh metrics
```

7.4 Monitoring Queries

Listing 7.3: PostgreSQL Monitoring Queries

```sql
-- Total revenue by bank today
SELECT
    bank_code,
    COUNT(DISTINCT customer_id) as customers,
    ROUND(SUM(total_amount_eur)::numeric, 2) as total_eur,
    SUM(transaction_count) as transactions
FROM gold.daily_customer_spending
WHERE transaction_date = CURRENT_DATE
GROUP BY bank_code
ORDER BY total_eur DESC;

-- High risk fraud alerts
SELECT
    transaction_id,
    customer_id,
    amount_eur,
    fraud_score,
    risk_level,
    is_large_amount,
    is_suspicious_ip,
    is_unknown_merchant,
    created_at
FROM gold.fraud_indicators
WHERE transaction_date = CURRENT_DATE
  AND risk_level = 'high'
ORDER BY fraud_score DESC, amount_eur DESC;

-- Top merchants by revenue today
SELECT
    merchant_name,
    merchant_category,
    ROUND(total_revenue_eur::numeric, 2) as revenue_eur,
    transaction_count,
    unique_customers
FROM gold.daily_merchant_summary
WHERE transaction_date = CURRENT_DATE
```

```sql
ORDER BY total_revenue_eur DESC
LIMIT 10;
```

8. Git Repository Structure

8.1 .gitignore

Listing 8.1: .gitignore

```
# Secrets - never commit
.env
.env.*
!.env.example

# Python
__pycache__/
*.pyc
.pytest_cache/

# Jupyter
.ipynb_checkpoints/

# Spark
spark-warehouse/
derby.log
metastore_db/

# Data files
data/sample/*.csv
data/sample/*.parquet

# OS
.DS_Store
```

8.2 Git Workflow

Listing 8.2: Initial Git Setup and Push

```
cd project-atlas

git init
git remote add origin https://github.com/yourname/project-atlas.git

# Stage everything except secrets
git add .
git status  # verify .env is NOT listed

git commit -m "feat: Project Atlas medallion pipeline

- Bronze streaming: Kafka -> Delta Lake (10s micro-batch)
```

```
- Silver streaming: validate, fraud signals, EUR normalisation
- Gold batch: aggregations -> PostgreSQL
- 12 Docker services: Kafka, Spark, MinIO, MongoDB, PostgreSQL
- Bank simulators: PKO, mBank, ING (~90 txns/min)
- Azure DevOps pipeline with 3-stage deploy
- Bicep IaC templates for Azure deployment"

git push -u origin main
```

9. Interview Reference

9.1 Key Talking Points

Bronze Layer

"Bronze stores everything exactly as received from the source system. All columns are StringType because Bronze accepts everything including bad data. We use Delta Lake for ACID transactions and time travel. If Silver has a bug we replay from Bronze without touching Kafka."

Silver Layer

"Silver validates every record against seven rules. Bad records go to a quarantine bucket with a reason column — we never silently drop data in a financial system. We compute five fraud signals per transaction and normalise all amounts to EUR for cross-bank comparison using a configurable FX rates dictionary."

Gold Layer

"Gold runs as a scheduled batch job every five minutes. We use a delete-first idempotent write pattern — delete today's records then append fresh aggregations. This means running Gold a hundred times produces the same result as running it once. Power BI connects directly to the PostgreSQL Gold tables."

Azure Migration

"The local code is identical to Azure. We swap Kafka for Event Hubs using the same Kafka protocol, MinIO paths change from s3a:// to abfss://, Spark runs on Databricks instead of local Docker, and secrets move from a .env file to Azure Key Vault accessed via dbutils.secrets.get(). The transformation logic in the notebooks is unchanged."

9.2 Architecture Decisions

Decision	Reason
All StringType in Bronze	Source cannot be trusted for types
foreachBatch in Silver	Write to multiple sinks per micro-batch
Quarantine not drop	Financial systems never lose records

EUR normalisation	Cross-bank comparability for analytics
UUID run IDs	Globally unique without central counter
Delete-first upsert	Idempotent writes without native UPSERT
Delta Lake checkpoints	Exactly-once streaming guarantees
MongoDB for audit	Sub-millisecond single-record writes
PostgreSQL for Gold	SQL interface for Power BI